VELLORE INSTITUTE OF TECHNOLOGY

BCSE307 — Compiler Design

Project Review 3 — Final Report

Bytecode JIT Compiler with Inline Caching

A stack-based bytecode VM with hidden classes, a four-state inline cache, a guarded tier-2 fast path with deoptimisation, and a JavaScript-subset front end verified against V8

Project ID: A4

S.No	Register No.	Name	Technical ownership
1	24BCT0093	Chilukuri Charith Reddy	VM & hot-path integration
2	24BCE0334	Vatsal Sharma	Profiler & hot-code detection
3	24BCE0506	Mayan Manit	Inline-cache integration & hit-rate measurement
4	24BCT0089	Ehsan Abdulkhader	Tier-2 fast path, deoptimisation, JS front end & evaluation

Team number: A4 Review date: 23 October 2026

Faculty / Evaluator: Shalini L

Repository: https://github.com/eak560/a4-bytecode-jit-inline-cache

Declaration

We, the undersigned members of Team A4, declare that this report and the accompanying software are our original work, carried out for the course BCSE307 Compiler Design at Vellore Institute of Technology. All external sources — books, papers, documentation and software tools — are acknowledged in the References section. No code was copied from other student teams. Results reported here were produced by running the scripts in the project repository; every timing and count is measured output, not an estimate.

Name	Register No.	Signature	Date
Chilukuri Charith Reddy	24BCT0093		
Vatsal Sharma	24BCE0334		
Mayan Manit	24BCE0506		
Ehsan Abdulkhader	24BCT0089		

Contents

1. Abstract **4**

2. Introduction and Motivation **5**

2.1 Context 5
2.2 Need and beneficiaries 5
2.3 Relevance to compiler design 5

3. Problem Statement, Objectives, Scope and Limitations **6**

3.1 Problem statement 6
3.2 Measurable objectives and compliance 6
3.3 Closure of Review 2 action items 6
3.4 Scope 7
3.5 Limitations (summary) 7

4. Background and Related Work **8**

4.1 Hidden classes (shapes) 8
4.2 Inline caches 8
4.3 Tiered compilation and deoptimisation 8
4.4 Comparison and identified gap 8

5. Requirements and Technology Stack **10**

5.1 Functional requirements 10
5.2 Non-functional requirements 10
5.3 Technology stack and environment 10
5.4 Test programs 10

6. System Design **11**

6.1 Architecture 11
6.2 Modules and interfaces 11
6.3 Core data structures 12
6.4 Execution paths of GETFIELD 12
6.5 Key design decisions 12

7. Algorithms and Methodology **14**

7.1 Bytecode execution and instruction set 14
7.2 Hot-loop detection 14
7.3 Inline-cache lookup and state transitions 14
7.4 Lazy invalidation by shape versioning 15
7.5 Tier-2 guarded load and deoptimisation 15
7.6 Front end: lexing, parsing and code generation 15
7.7 Complexity and trade-offs 16

8. Implementation Details **17**

8.1 Code organisation 17
8.2 Module walkthrough 17
8.3 Example: source to bytecode 18
8.4 Build and integration 18

9. Testing Strategy and Test Cases **19**

9.1 Strategy 19
9.2 Unit tests 19
9.3 End-to-end tests against Node.js 20
9.4 Invalid-input and failure tests 20
9.5 Summary 21

10. Results and Evaluation **22**

10.1 Benchmark setup 22
10.2 Synthetic workloads (200-field objects) 22

10.3 Megamorphic cache tuning 22
10.4 JavaScript programs 23
10.5 Hot-loop threshold sweep 24
10.6 Comparison with V8 25
10.7 Overall assessment 25

11. Error Handling, Limitations and Known Issues 26
11.1 Error handling 26
11.2 Unsupported features 26
11.3 Known behavioural differences from JavaScript 26
11.4 Technical constraints 26

12. Member-Wise Contribution 28

13. Conclusion and Future Work 29
13.1 Objectives achieved 29
13.2 Key learning 29
13.3 Future work 29
13.4 Publication and further-development potential 29

14. References 30

Appendix A: Installation and User Manual 31
A.1 Requirements 31
A.2 Getting the code 31
A.3 Commands 31
A.4 Writing your own program 31

Appendix B: Sample Runs 32
B.1 run_js.py on examples/deopt.js 32
B.2 Invalid input 32

Appendix C: Grammar of the Supported JavaScript Subset 33

Appendix D: Complete Test Table 34

Appendix E: Contribution Log (Repository History) 36

1. Abstract

Dynamically typed languages such as JavaScript look up object properties by name at run time, which makes the simplest implementation — scanning the property list on every access — slow. Production engines such as V8 solve this with hidden classes, inline caches and tiered just-in-time (JIT) compilation with deoptimisation. This project builds a complete, measurable reference implementation of those ideas. It consists of (i) a stack-based bytecode virtual machine with a 15-opcode instruction set and back-edge loop profiling; (ii) hidden classes ("shapes") with version numbers for lazy invalidation; (iii) a per-call-site inline cache that moves through the UNINIT, MONO, POLY and MEGA states; (iv) a tier-2 path that compiles hot monomorphic property reads into guarded direct loads and deoptimises safely when a guard fails; and (v) jsc.py, a single-pass recursive-descent compiler from a JavaScript subset to the VM's bytecode.

Correctness is established by differential testing: every program runs on the baseline interpreter, with the inline cache, and with the cache plus tier 2, and all outputs are compared with each other and with Node.js (V8). The regression suite passes 34 of 34 checks, covering unit, end-to-end, invalid-input and stress cases. On 200-field objects the inline cache is 2.09× faster than the baseline for monomorphic sites and 1.49× for polymorphic sites (median of 5 runs); on a 16-shape megamorphic site with 4 slots it is 0.95×. A capacity/eviction sweep shows that capacity, not eviction policy, fixes the megamorphic case (16 slots: 100% hits, 1.48×). On small 6-field objects the cache costs more than the scan it replaces, while tier 2 recovers the loss — an honest result that mirrors why real engines compile cached accesses to machine code.

2. Introduction and Motivation

2.1 Context

A compiler for a statically typed language knows, when it compiles `p.x`, exactly where field `x` sits in memory, and can emit a single load instruction. A JavaScript engine does not: objects are created and extended at run time, and the same line of code may see objects of many different layouts. The naive implementation treats every object as a dictionary and looks the property up by name on every access. Because property access is among the most frequent operations in real programs, this cost dominates execution time.

The techniques that make modern JavaScript fast were developed over several decades of research on Smalltalk and Self and are now standard in engines such as V8 (Chrome, Node.js), SpiderMonkey (Firefox) and JavaScriptCore (Safari): objects share hidden classes that describe their layout; each property-access site keeps an inline cache that remembers the layouts it has seen; and frequently executed code is recompiled into fast, specialised code that relies on assumptions, with deoptimisation as the safety net when an assumption breaks.

2.2 Need and beneficiaries

These mechanisms are usually taught as diagrams; real engines are millions of lines of C++ in which they are hard to observe. A small, complete and instrumented implementation lets students and instructors see each mechanism working, measure its effect, and break it deliberately to understand why it is built the way it is. The beneficiaries are compiler-design students, instructors who need a runnable teaching artefact, and developers who want an intuition for why their JavaScript code is fast or slow (for example, why constructing objects with a consistent property order matters).

2.3 Relevance to compiler design

The project touches most stages of a compiler and runtime: lexical analysis (a regular-expression lexer), syntax analysis (a recursive-descent parser for an expression grammar with precedence), intermediate representation (stack bytecode), code generation with back-patching of jump targets, lowering of operators to a restricted instruction set, interpretation (fetch–decode–execute), run-time profiling, run-time code specialisation (tier 2) and speculative optimisation with deoptimisation. It therefore connects the front-end theory of the course with the run-time techniques used in production compilers.

3. Problem Statement, Objectives, Scope and Limitations

3.1 Problem statement

Design, implement and evaluate a bytecode execution engine that accelerates dynamic property access using hidden classes, inline caching and a guarded tier-2 fast path with deoptimisation, such that every optimised execution produces exactly the same output as an unoptimised reference interpreter — and demonstrate it on real JavaScript programs whose output is checked against a production engine.

3.2 Measurable objectives and compliance

Table 1: *Final objective-compliance matrix*

#	Objective (from Review 1)	Measure	Status	Evidence
O1	Stack-based VM and bytecode format	All opcodes implemented and tested	Met	15 opcodes; TC1–TC12 pass
O2	Baseline interpreter as correctness oracle	All baseline tests pass	Met	TC1–TC12, 12/12
O3	Hot-code detection	Hot loop flagged at threshold; threshold configurable	Met	TC11; VM(hot_threshold); threshold_sweep.py
O4	Inline caching (mono/poly/mega) with invalidation	Each state reached; stale entries rejected	Met	IC1–IC4; mega_tuning.py
O5	Tier-2 fast path with deoptimisation	Guarded load used; deopt resets IC; 0 wrong answers	Met (model)	IC5: 1 deopt, 92 fast-path reads
O6	Benchmarking and differential testing	0 mismatches across workloads; speedup measured	Met	34/34 checks; benchmark.py
O7	Extension: JavaScript front end	Programs compiled and matched against Node	Met	jsc.py; 7 programs × 3 modes = Node

O5 is met in the reference implementation: the guarded fast path and deoptimisation are implemented, executed and tested in Python. Emitting the same guards as native machine code (for example with asmjit) is outside the delivered scope and is listed as future work (Section 13).

3.3 Closure of Review 2 action items

Review 2 ended with six next steps. Their final status is:

Table 2: *Review 2 next steps and their closure*

Review 2 next step	Outcome in Review 3	Evidence
Exercise the deoptimisation path end to end	Closed — deopt runs inside the VM loop, resets the site to UNINIT and recompiles	IC5; examples/deopt.js matches Node
Tune the megamorphic cache	Closed — capacity × policy sweep; capacity fixes it	mega_tuning.py, Section 10.3
Make the hot threshold configurable and sweep it	Closed — VM(hot_threshold=…); swept 1–100	threshold_sweep.py, Section 10.5

Review 2 next step	Outcome in Review 3	Evidence
Run the full benchmark for final numbers	Closed — median of 5 runs on one machine	benchmark.py, Section 10
Draft the final report	Closed — this document	—
Emit IC guards in native code	Not delivered — future work	Section 13

3.4 Scope

- **In scope:** the bytecode VM and profiler; hidden classes with versioning; the inline-cache state machine; tier-2 guarded loads and deoptimisation; a compiler for a JavaScript subset; differential testing against Node; benchmarks and parameter sweeps.
- **Out of scope:** generation of native machine code; garbage collection; the full JavaScript language (strings, functions, closures, prototypes, property writes).

3.5 Limitations (summary)

The engine is a Python reference model, so its timings show algorithmic savings (fewer comparisons per property read), not machine-code speed. The JavaScript subset is deliberately small. Section 11 lists every unsupported construct and known behavioural difference from JavaScript.

4. Background and Related Work

4.1 Hidden classes (shapes)

Self [3] introduced *maps*: objects created with the same properties in the same order share one map that records the offset of each property, so an object needs to store only its values plus a pointer to its map. V8 calls these hidden classes or *shapes* and builds them as transition trees as properties are added [9]. Two objects `{x, y}` and `{y, x}` therefore have different shapes, because `x` is at a different offset.

4.2 Inline caches

Deutsch and Schiffman [2] observed that a given call site in Smalltalk usually sees receivers of the same class, and cached the lookup result directly at the call site — an *inline cache*. Hölzle, Chambers and Ungar [4] generalised this to *polymorphic inline caches* (PICs) that hold a short list of (class, target) pairs, and showed that sites with too many classes (megamorphic sites) should fall back to a slower generic lookup. Modern engines use exactly this monomorphic / polymorphic / megamorphic classification; V8 caps polymorphic caches at four entries and backs megamorphic sites with a global shared cache [9].

4.3 Tiered compilation and deoptimisation

A JIT compiler [6] starts in an interpreter, profiles which code is hot and which types it sees, and recompiles hot code into specialised machine code. Specialisation is speculative: the code assumes, for example, that an object has a particular shape. Hölzle, Chambers and Ungar [5] introduced *dynamic deoptimisation*, which transfers execution back to unoptimised code when an assumption fails. Trace-based JITs such as TraceMonkey [7] and partial-evaluation systems such as Truffle/Graal [8] use the same guard-and-deoptimise pattern.

4.4 Comparison and identified gap

Table 3: *Comparison with production engines and textbook coverage*

Aspect	V8 / production engines	Compiler textbooks [1]	This project
Hidden classes	Yes (transition trees)	Not covered	Yes (interned shapes + version)
Inline caches	Mono/poly/mega + shared cache	Not covered	Mono/poly/mega, bounded table
Tier-2 + deopt	Optimising compiler (TurboFan)	Not covered	Guarded direct load + deopt
Front end	Full JavaScript	Covered in theory	JS subset, recursive descent
Observability	Hard (millions of LOC)	—	Every hit, miss, compile, deopt counted
Correctness evidence	Large test suites	—	Differential vs baseline and vs V8

The gap this project addresses is a *small, complete and measurable* implementation in which each of these mechanisms can be executed, counted and compared, with the production engine used as the correctness reference.

5. Requirements and Technology Stack

5.1 Functional requirements

1. Execute stack bytecode correctly, including arithmetic, locals, jumps, comparison, output and property access.
2. Detect hot loops by counting backward jumps.
3. Cache property offsets per access site and transition through UNINIT, MONO, POLY and MEGA.
4. Never return a stale offset after a shape changes.
5. Compile hot monomorphic sites to guarded loads and deoptimise on guard failure without changing output.
6. Compile a JavaScript subset to bytecode and report clear errors for invalid or unsupported input.
7. Check every optimised run against the baseline and against Node.js.

5.2 Non-functional requirements

- **Reproducibility:** fixed random seeds; single-command test and benchmark scripts; no third-party Python packages.
- **Observability:** every hit, miss, compile, fast-path read and deopt is counted.
- **Safety:** a step limit stops runaway programs; errors are reported, never silently wrong.

5.3 Technology stack and environment

Table 4: *Software, hardware and test environment*

Item	Version / detail	Role
Python	3.12.2	VM, compiler, tests, benchmarks (standard library only)
Node.js / V8	v26.0.0 / V8 14.6.202.33-node.19	Reference output; V8 comparison
Operating system	macOS 26.5.2	Test and benchmark machine
Hardware	Apple M3 Pro	Apple Silicon laptop, 18 GB RAM
matplotlib	3.9.2	Report figures only (not needed to run the project)
Git / GitHub	public repository	Version control and submission

5.4 Test programs

Seven JavaScript programs in `examples/` (language features, monomorphic, polymorphic, megamorphic and wide-object workloads, a deoptimisation scenario and boundary cases) and seven invalid programs in `tests/invalid/`. The synthetic benchmarks generate objects with 200 filler fields using fixed seeds (42 for benchmarks, 7 for stress tests).

6. System Design

6.1 Architecture

Figure 1 shows the final architecture. A JavaScript file is compiled by `jsc.py` into bytecode plus a table of pre-built objects. The runtime in `vm_ic_model.py` executes it; the profiler marks hot loops; property reads go through the inline cache and, when hot and monomorphic, through tier 2. Node.js runs the same file, and the outputs are compared line by line.

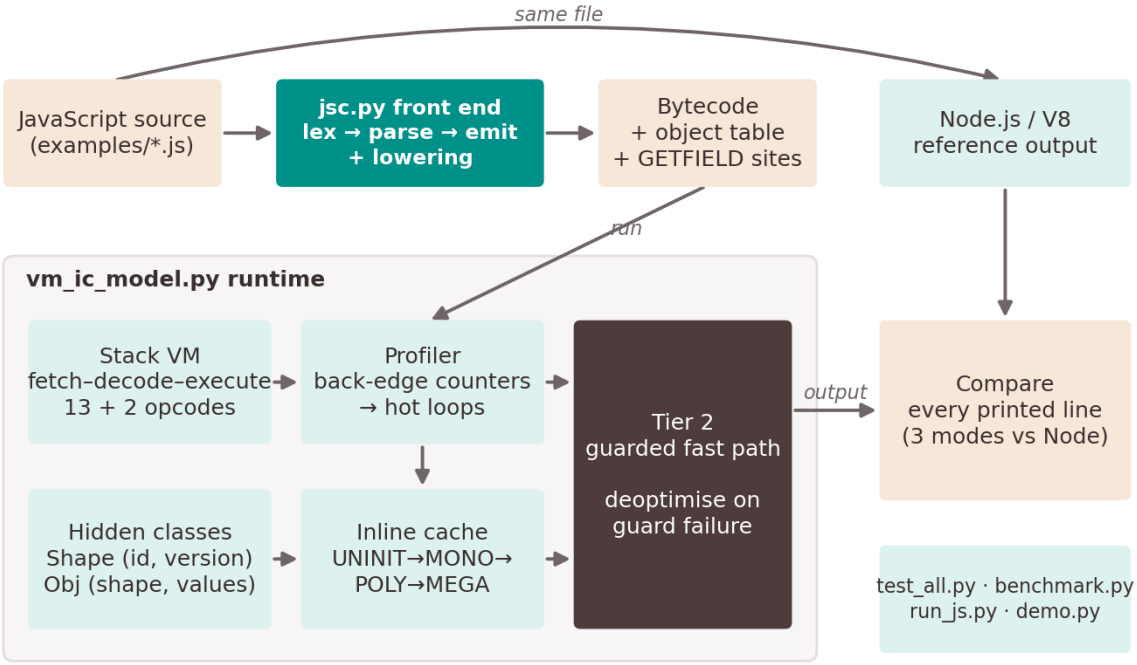


Figure 1: *Final architecture and data flow*

6.2 Modules and interfaces

Table 5: *Modules, responsibilities and interfaces*

Module	File / entry point	Input → output	Owner
Front end (lexer, parser, code generator)	jsc.py · compile_js(src)	JS text → (bytecode, compiler state)	Member 4
Bytecode VM / interpreter	vm_ic_model.py · VM.run()	bytecode → output list	Member 1
Profiler (hot-loop detection)	VM.run() back-edge counters	jumps → hot_flags	Member 2
Hidden classes	Shape, Obj, make_object, mutate_shape	property dict → shaped object	Member 3
Inline cache	ICEntry, ic_getprop()	(site, object, name) → value	Member 3
Tier 2 and deoptimisation	VM.compiled, VM.deoptimize()	hot MONO site → guarded load	Member 4
Evaluation harness	run_js.py, test_all.py, benchmark.py, mega_tuning.py, threshold_sweep.py	programs → reports	Member 4

6.3 Core data structures

- **Instruction:** a tuple `(op, arg)`, e.g. `("GETFIELD", "x")`. A program is a list of instructions; the program counter `pc` indexes it.
- **Shape:** `id`, ordered `props` list and `version`. Shapes are interned in a registry keyed by the property-name tuple, so equal layouts share one shape object.
- **Obj:** a `shape` reference and a parallel `values` list (`__slots__` to keep it compact).
- **ICEntry (one per GETFIELD site, keyed by pc):** `state`, `entries` — a list of `(shape_id, version, offset)` for MONO/POLY — and `mega_table`, an ordered dictionary bounded by `MEGA_CACHE_CAPACITY`.
- **Tier-2 table:** `VM.compiled`, a map from pc to `(shape_id, version, offset)` — the guard and the offset of the compiled load.

6.4 Execution paths of GETFIELD

Property reads are the focus of every optimisation. Figure 2 shows the three paths a GETFIELD can take, chosen per site.

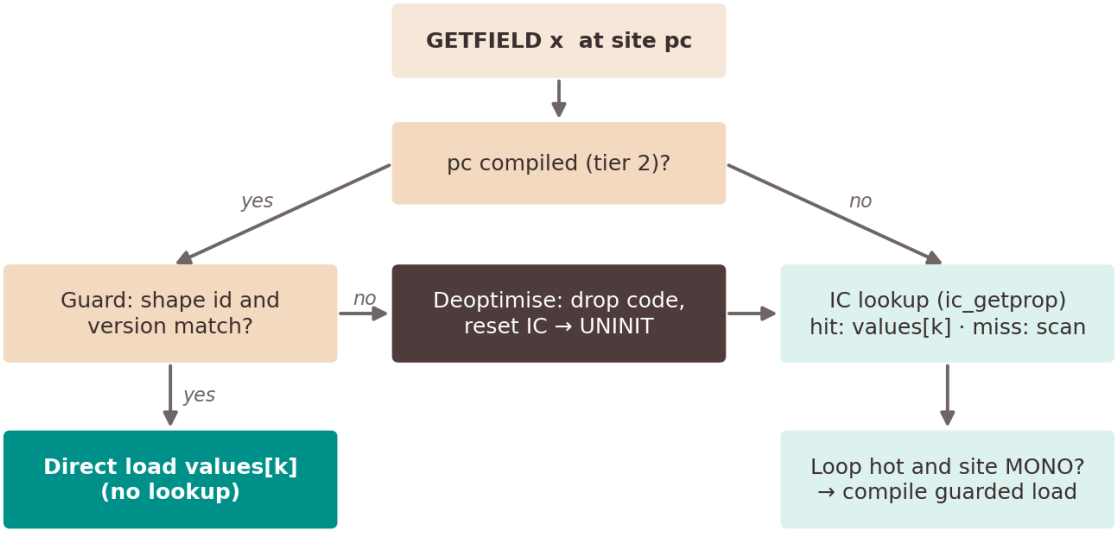


Figure 2: *The three execution paths of a GETFIELD instruction*

6.5 Key design decisions

Table 6: *Design decisions and their justification*

Decision	Alternative considered	Justification
Stack-based bytecode	Register-based bytecode	Simpler code generation (no register allocation); used by the JVM, CPython and WebAssembly
Cache keyed by bytecode address (pc)	One global cache	Matches the *inline* in inline cache: sites behave differently and must not pollute each other
Version number on shapes, checked on use	Flush every cache on mutation	Lazy invalidation is O(1) per access and needs no list of dependent caches
Hard reset to UNINIT on deopt	Append the new shape	A failed speculation means the site's history is unreliable; resetting avoids repeated wrong guesses

Decision	Alternative considered	Justification
Compile only MONO sites	Compile POLY sites as guard chains	One guard keeps the fast path minimal and the deopt logic simple
Single-pass compiler with back-patching	Build an AST, then generate code	Fewer data structures; jump targets are patched once known
Bounded MEGA table	Unbounded dictionary	Models real size-limited caches so megamorphic behaviour can be studied
Python standard library only	C/C++ with asmjit	Fast iteration and full instrumentation; correctness first, native code as future work

7. Algorithms and Methodology

7.1 Bytecode execution and instruction set

The VM is a fetch–decode–execute loop over a value stack, an array of local slots and an object table. Each instruction is dispatched by opcode; a step counter raises an error if a program exceeds `max_steps`, so infinite loops terminate safely.

Table 7: *Instruction set (15 opcodes)*

Group	Opcodes	Stack effect
Stack	PUSH v, POP	push constant; discard top
Arithmetic	ADD, SUB, MUL, DIV	pop b, pop a, push a∘b (DIV is integer; ÷0 raises VMError)
Locals	LOAD k, STORE k	push local k; pop into local k
Control	JMP t, JZ t, LT	jump; pop and jump if 0; pop b, a, push (a<b ? 1 : 0)
I/O	PRINT, HALT	record top of stack; stop
Objects	LOADOBJ, GETFIELD name	pop index, push object; pop object, push property value

7.2 Hot-loop detection

A backward jump (target ≤ current pc) is a loop back-edge. The VM increments a counter for the target; when it reaches `hot_threshold` the loop head is added to `hot_flags`. Counting only back-edges costs one comparison per jump and identifies exactly the code that repeats. TC11 verifies it; Section 10.5 sweeps the threshold.

7.3 Inline-cache lookup and state transitions

Figure 3 shows the state machine of one site. The lookup algorithm treats an entry as a hit only if both the shape id and the shape version match:

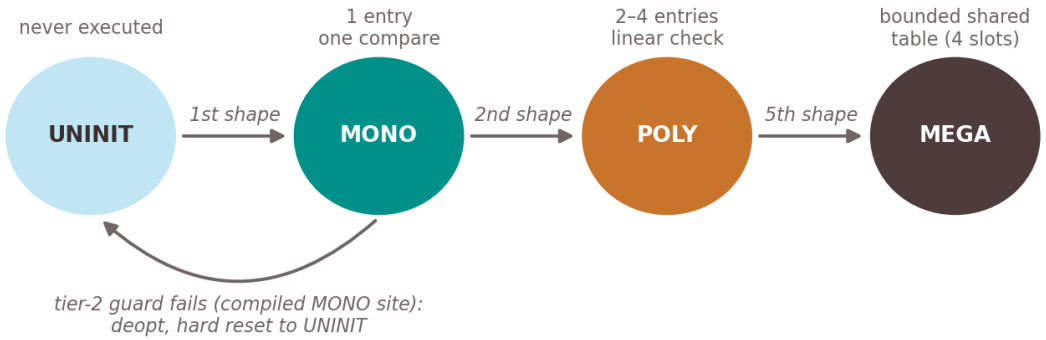


Figure 3: *Inline-cache state machine of one GETFIELD site*

```
ic_getprop(cache, obj, name):
    if cache.state == MEGA:
        entry = cache.mega_table.get(obj.shape.id)
        if entry and entry.version == obj.shape.version: return hit
        offset = linear_scan(obj, name); insert; evict oldest (FIFO) or LRU if full
    for (sid, ver, off) in cache.entries:              # at most 4
        if sid == obj.shape.id and ver == obj.shape.version: return obj.values[off]
    offset = linear_scan(obj, name)                    # miss
    replace any stale entry for this shape; append (id, version, offset)
    state = MONO if 1 entry, POLY if 2-4, else move entries into mega_table -> MEGA
```

7.4 Lazy invalidation by shape versioning

`mutate_shape` inserts a property into an existing shape in place (modelling a class gaining a field after objects exist), re-lays-out every object of that shape and increments `shape.version`. Nothing else is updated. Any cache entry recorded under the old version now fails the version comparison and is re-resolved on its next use. IC4 demonstrates that without this check the cache would return the wrong value (2 instead of 3).

7.5 Tier-2 guarded load and deoptimisation

After a GETFIELD is resolved by the cache, if any loop is hot and the site is MONO, the VM records `compiled[pc] = (shape_id, version, offset)`. Later executions of that site skip the cache entirely:

```
if pc in compiled:
    sid, ver, off = compiled[pc]
    if obj.shape.id == sid and obj.shape.version == ver:   # the guard
        value = obj.values[off]                            # direct load
    else:                                                  # deoptimise
        del compiled[pc]; cache.reset() -> UNINIT
        value = ic_getprop(cache, obj, name)               # safe re-execution
```

Because the deoptimised access is re-executed through the cache and interpreter, the result is identical to the baseline. After a deopt the site may become MONO again and be recompiled for the new shape (IC5 compiles twice).

7.6 Front end: lexing, parsing and code generation

Lexing. One regular expression with named groups recognises whitespace and comments, integers, identifiers and operators (longest operators first, e.g. `===` before `==`). Each token records its line number for error messages.

Parsing. A hand-written recursive-descent parser implements the grammar in Appendix C, one function per precedence level (comparison < additive < multiplicative < unary < postfix < primary). Code is emitted during parsing; there is no separate AST.

Back-patching. For `if`, `while` and `for`, the parser emits `JZ` with an unknown target, records its index, compiles the body and then patches the target. A `for` loop's update clause is skipped and compiled after the body by re-parsing its token range.

Lowering. The instruction set has only `LT`, so other operators are rewritten using compiler-generated temporary locals (`$tN`):

Table 8: *Lowering of operators not in the instruction set*

Source	Lowered form	Instructions
`a < b`	`a < b`	`LT`
`a > b`	`b < a`	`store a, b in temps; LOAD b, LOAD a, LT`
`a >= b`	`1 − (a < b)`	`PUSH 1, (a<b), SUB`
`a == b`	`1 − ((a<b) + (b<a))`	`two LTs, ADD, SUB`
`a != b`	`(a<b) + (b<a)`	`two LTs, ADD`
`a % b`	`a − (a / b) × b`	`LOAD a, LOAD a, LOAD b, DIV, LOAD b, MUL, SUB`
`!a`	`a == 0`	`PUSH 0, equality lowering`
`-a`	`0 − a`	`PUSH 0, a, SUB`

Literals. Object and array literals contain only constants, so they are built at compile time into the VM's object table; `a[i]` compiles to `PUSH base, i, ADD, LOADOBJ`. This is safe because the subset has no property or element writes.

7.7 Complexity and trade-offs

Table 9: *Cost of one property read (n = number of properties, k = cached entries ≤ 4)*

Path	Time per read	Space	Trade-off
Baseline scan	O(n)	none	Simple and always correct; slow for wide objects
IC hit, MONO	O(1)	1 entry per site	Bookkeeping cost per read even when n is small
IC hit, POLY	O(k), k ≤ 4	≤ 4 entries per site	Short list stays cheap
IC, MEGA	O(1) hit / O(n) miss	capacity entries per site	Thrashes when shapes > capacity
Tier 2	O(1), one guard	1 entry per site	Only for hot MONO sites; deopt cost on failure
Compiler	O(tokens)	O(code size)	Single pass; no AST to optimise over

8. Implementation Details

8.1 Code organisation

Table 10: *Repository files and size (lines, including comments)*

File	Lines	Purpose
`vm_ic_model.py`	622	VM, profiler, hidden classes, inline cache, tier 2 + deopt, TC/IC tests, synthetic benchmark
`jsc.py`	400	JavaScript-subset compiler: lexer, recursive-descent parser, code generation, lowering
`run_js.py`	150	Runs a .js file in Node and on the VM in 3 modes; compares output; per-site cache report
`test_all.py`	108	Regression suite: unit, end-to-end vs Node, invalid input, stress
`benchmark.py`	122	Median-of-N benchmark for the report (sections A–D)
`mega_tuning.py`	36	Megamorphic capacity × eviction-policy sweep
`threshold_sweep.py`	42	Hot-loop threshold sweep
`demo.py`	185	Paced presentation walkthrough (7 beats)
`v8_compare.js`	49	Same mono/poly/mega experiment inside V8
`run_demo.command`	29	One-click live demo
`examples/*.js`	190	7 JavaScript test programs
`tests/invalid/*.js`	18	7 invalid / failure programs

8.2 Module walkthrough

VM and profiler (vm_ic_model.py, VM.run)

A single `while pc < len(program)` loop with an `if/elif` chain on the opcode. JMP and JZ update `back_edge_counts` and `hot_flags` when the target is backward. GETFIELD delegates to the tier-2 table, the inline cache or the baseline scan depending on the mode flags passed to `run()`.

Hidden classes and the inline cache

`Shape.__new__` interns shapes by property-name tuple. `ic_getprop` implements Section 7.3; the MEGA table is an `OrderedDict` whose eviction order is FIFO by default and LRU when `MEGA_POLICY = "LRU"` (hits call `move_to_end`). Both parameters are module globals read at call time so the sweep scripts can vary them without changing code.

Tier 2 and deoptimisation

`VM.compiled` maps a pc to its guard and offset. `VM.deoptimize()` deletes the entry, calls `cache.__init__()` to hard-reset the site, records the post-reset state for IC5, and redoes the access through `ic_getprop`.

Front end (jsc.py)

`tokenize()` produces `(kind, text, line)` tuples. The `Compiler` class holds the token cursor, emitted code, the variable-to-slot map, array descriptors, the object table and a map from each

GETFIELD pc to its source line, which `run_js.py` uses to report cache state per source line. Unsupported keywords (`function`, `return`, `true`, …) raise a `CompileError` with the line number.

Evaluation harness

`run_js.py` runs Node with a 10-second timeout, compiles the file, runs it in three modes, compares every line and prints per-site cache states. `test_all.py` is the regression suite (Section 9). `benchmark.py` repeats each timing five times and reports medians. `mega_tuning.py` and `threshold_sweep.py` are the parameter sweeps; `run_demo.command` is the one-click demo.

8.3 Example: source to bytecode

The loop body `total += items[i].x;` from `examples/deopt.js` compiles to the following bytecode; address 13 is the site that the cache and tier 2 optimise:

```
 4  LOAD 1         loop head: i
 5  PUSH 20        items.length (compile-time constant)
 6  LT             i < length ?
 7  JZ 21          exit loop (target back-patched)
 8  LOAD 0         total
 9  PUSH 0         base address of items in the object table
10  LOAD 1         i
11  ADD            base + i
12  LOADOBJ        items[i]
13  GETFIELD x     .x   <- inline-cache / tier-2 site
14  ADD            total + ...
15  STORE 0        total = ...
```

8.4 Build and integration

There is no build step: the project is pure Python 3 with no third-party dependencies. Integration is enforced by the test suite, which exercises the front end, VM, cache and tier 2 together on every example and compares the result with Node. The full procedure is in Appendix A.

9. Testing Strategy and Test Cases

9.1 Strategy

Testing has four layers, all run by `python3 test_all.py`:

1. **Unit tests** — TC1–TC12 for the interpreter and IC1–IC5 for the inline cache, invalidation and deoptimisation. Each is a tiny program with a known answer.
2. **Differential end-to-end tests** — each JavaScript example runs in three modes; all must equal Node's output line for line. This catches errors in the compiler and in every optimisation at once.
3. **Negative and failure tests** — invalid programs must be rejected at compile time, or fail at run time, with a specific diagnostic.
4. **Stress tests** — 50,000-access mono, poly and mega workloads with 0 mismatches between the cached and baseline runs.

Boundary cases are covered by TC6 (division by zero), TC10 (equal operands for LT), TC12 (1,000-deep stack) and `examples/edge_cases.js` (zero-iteration loop, negative and large integers, four chained property reads, single-element array). Regression protection comes from running the whole suite after every change.

9.2 Unit tests

Table 11: *Interpreter unit tests (TC1–TC12)*

Test	Operation	Expected	Actual
TC1	10 + 20	30	30 — PASS
TC2	20 − 5	15	15 — PASS
TC3	10 × 5	50	50 — PASS
TC4	20 ÷ 4	5	5 — PASS
TC5	(10 + 20) × 5	150	150 — PASS
TC6	10 ÷ 0	VMError	VMError — PASS
TC7	LOAD/STORE in 4 slots	11, 22, 33, 44	11, 22, 33, 44 — PASS
TC8	JMP over PRINT	[1]	[1] — PASS
TC9	JZ taken / not taken	[7] and [42]	PASS
TC10	LT on <, >, =	1, 0, 0	PASS
TC11	10-iteration loop, threshold 5	address 2 flagged hot	back-edges {2: 10} — PASS
TC12	1,000 PUSH then 1,000 POP	empty stack	empty — PASS

Table 12: *Inline-cache unit tests (IC1–IC5)*

Test	Scenario	Expected	Actual
IC1	1 shape, 1,000 reads	MONO, 1 miss, 0 mismatches	MONO, 1 miss, 0 mismatches
IC2	3 shapes round-robin	POLY, 3 entries, 3 misses	POLY, 3 entries, 3 misses
IC3	8 shapes round-robin	MEGA, 0 mismatches	MEGA, 0 hits / 1,000 misses, 0 mismatches

Test	Scenario	Expected	Actual
IC4	Shape mutated between runs	version bump, 1 re-miss, correct value	v0→v1; stale slot reads 2 (not 3); 1 re-miss
IC5	50 shape-A then 50 shape-B, tier 2	1 deopt, reset to UNINIT, 0 mismatches	2 compiles, 92 fast-path reads, 1 deopt, UNINIT

9.3 End-to-end tests against Node.js

Table 13: *JavaScript programs: every mode compared with Node*

Program	What it exercises	Lines compared	Result
basics.js	Precedence, if/else, while, for, %, objects, arrays	14 lines	3 modes = Node
deopt.js	Shape change mid-loop; tier-2 deopt	20 lines	3 modes = Node
edge_cases.js	Zero-iteration loop, negatives, 2^40, deep chains	9 lines	3 modes = Node
points_mega.js	16 shapes at one site	1 lines	3 modes = Node
points_mono.js	1 shape, 96,000 property reads	1 lines	3 modes = Node
points_poly.js	3 shapes (field order permuted)	1 lines	3 modes = Node
points_wide.js	1 shape, 32 fields	1 lines	3 modes = Node

9.4 Invalid-input and failure tests

Table 14: *Negative tests: expected diagnostic vs actual*

Program	Expected	Actual diagnostic	Result
division_by_zero.js	runtime error (division by zero)	division by zero	PASS
infinite_loop.js	runtime error (step limit exceeded)	step limit exceeded (possible infinite loop)	PASS
missing_property.js	runtime error (property 'y' not found)	property 'y' not found on object	PASS
syntax_error.js	compile error (expected ')')	line 2: expected ')', got ';'	PASS
undefined_variable.js	compile error ('y' is not defined)	line 2: 'y' is not defined	PASS
unsupported_function.js	compile error ('function' is not supported)	line 2: 'function' is not supported by this compiler	PASS
unsupported_string.js	compile error (unexpected character)	line 2: unexpected character '"'	PASS

Three of these behave differently in JavaScript, which `run_js.py` reports explicitly: Node prints `Infinity` for `10 / 0` and `undefined` for a missing property, where our VM raises an error; and Node never terminates the infinite loop (it is stopped by a 10-second timeout), whereas our VM stops it at the step limit. These are documented in Section 11.

9.5 Summary

Regression suite result: 34 / 34 checks pass — 17 unit, 7 end-to-end, 7 invalid-input and 3 stress checks. The complete table is in Appendix D.

10. Results and Evaluation

10.1 Benchmark setup

All timings were measured on one machine (Apple M3 Pro, 18 GB RAM, macOS 26.5.2, Python 3.12.2) by `python3 benchmark.py`, which runs each experiment 5 times and reports the median. Speedup is baseline time divided by optimised time; values above 1× are faster. Hit rates and mismatch counts are deterministic because the workloads use fixed seeds. The timings measure a Python model and therefore reflect algorithmic savings, not native performance.

10.2 Synthetic workloads (200-field objects)

Each workload performs 100,000 GETFIELD reads at one site inside a bytecode loop. The target property sits after up to 200 other properties, so the uncached scan is expensive.

Table 15: *Inline cache vs baseline, 100,000 reads (median of 5 runs)*

Workload	Hits / misses	Hit rate	Baseline (s)	IC (s)	Speedup	Mismatches
Monomorphic (1 shape)	99,999 / 1	100.0%	0.374	0.177	2.09×	0
Polymorphic (3 shapes)	99,997 / 3	100.0%	0.269	0.182	1.49×	0
Megamorphic (16 shapes, 4 slots)	24,932 / 75,068	24.9%	0.266	0.278	0.95×	0

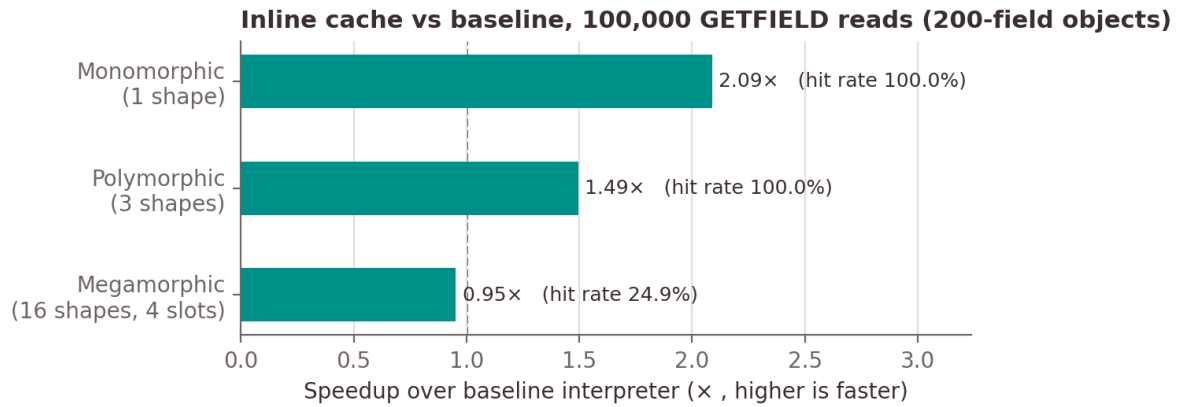


Figure 4: *Speedup of the inline cache on 200-field objects*

Interpretation. A monomorphic hit replaces a scan over up to 200 names with one comparison, giving 2.09×. Polymorphic sites check a list of three entries and some shapes have shorter scans, giving 1.49×. The megamorphic site thrashes: 16 shapes compete for 4 slots, only 24.9% of reads hit, and the bookkeeping cancels the savings (0.95×). Correctness is unaffected — 0 mismatches in every run.

10.3 Megamorphic cache tuning

To close the Review 2 action item, the megamorphic table was swept over capacity (4, 8, 16 slots) and eviction policy (FIFO, LRU) on the same 16-shape workload.

Table 16: *Megamorphic capacity × eviction policy (median of 5 runs)*

Capacity	Policy	Hit rate	Speedup	Mismatches
4 slots	FIFO	24.9%	0.96×	0
4 slots	LRU	25.0%	0.97×	0
8 slots	FIFO	49.9%	1.09×	0
8 slots	LRU	49.9%	1.09×	0
16 slots	FIFO	100.0%	1.48×	0
16 slots	LRU	100.0%	1.47×	0

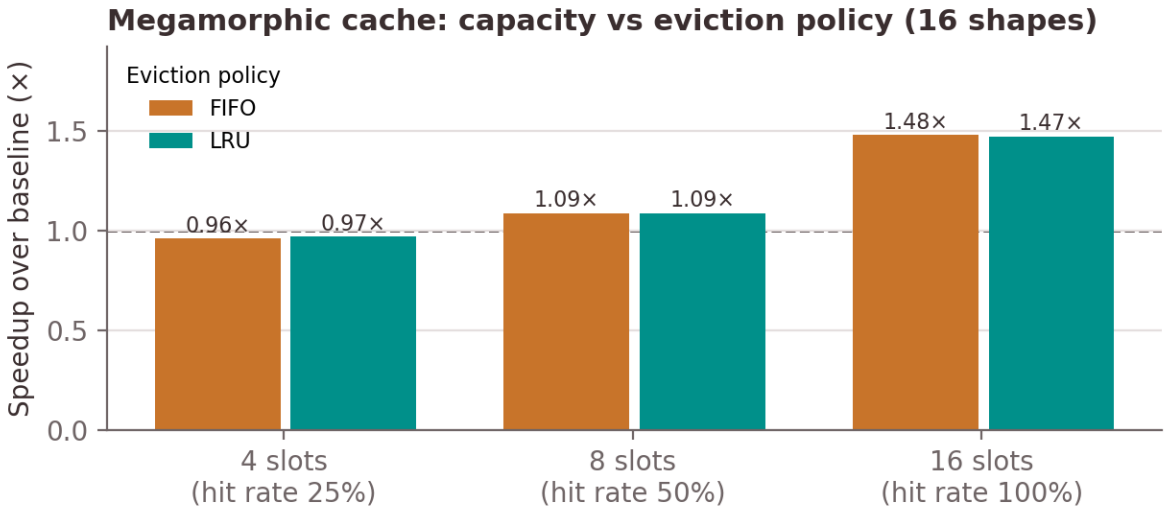


Figure 5: *Megamorphic cache: speedup by capacity and eviction policy*

Interpretation. Hit rate tracks capacity ÷ number of shapes (≈25%, 50%, 100%), and FIFO and LRU reach the *same* hit rate at every capacity (24.9% vs 25.0%; 49.9% vs 49.9%; 100.0% vs 100.0%). With shapes arriving in random order every shape is equally likely to be needed next, so no eviction policy can choose a better victim: the policy does not change how often the cache misses. Their timings differ by at most 0.01×. With 16 slots the site never evicts, reaches 100% hits and is 1.48× (FIFO) / 1.47× (LRU) faster than the baseline. The trade-off is memory per site, which is why production engines give each site a small cache backed by one large shared megamorphic cache.

10.4 JavaScript programs

The JavaScript examples use small objects (6 fields) except `points_wide.js` (32 fields), which makes the cost of the cache itself visible.

Table 17: *JavaScript programs: speedup over our baseline (median of 5 runs)*

Program	Hit rate	IC	IC + tier 2	Fast-path reads
points_mono.js	100.0%	0.89×	1.14×	95,976
points_poly.js	100.0%	0.85×	0.84×	0
points_mega.js	0.0%	0.66×	0.68×	0
points_wide.js	100.0%	1.17×	1.50×	95,976

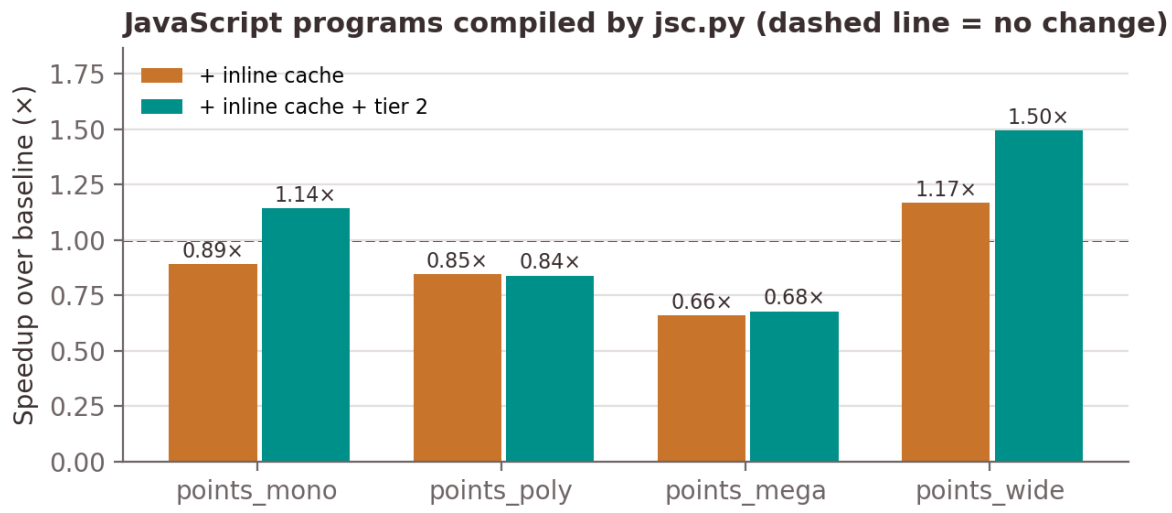


Figure 6: *Speedup of the cache and tier 2 on the JavaScript examples*

Interpretation. With 6 fields the scan is so short that the cache's bookkeeping costs more than it saves: the cache alone is 0.89× on `points_mono` despite a 100% hit rate. Tier 2 removes the cache lookup for hot monomorphic sites and recovers the loss (1.14×). Polymorphic and megamorphic sites are never compiled (only MONO sites are), so tier 2 cannot help them. With 32 fields the scan is longer: the cache alone wins (1.17×, against 0.89× with 6 fields) and tier 2 reaches 1.50×. The cache pays off in proportion to the cost of the lookup it replaces — the reason production engines compile cached accesses into machine code.

10.5 Hot-loop threshold sweep

The threshold decides when a loop is hot and its monomorphic sites are compiled. The table and figure below show its effect.

Table 18: *Hot-threshold sweep: compiles, tier-2 reads, deopts, mismatches*

Threshold	IC5: compiles	IC5: fast-path reads	IC5: deopts	Mono: fast-path reads	Mismatches
1	2	96	1	198	0
2	2	95	1	197	0
5	2	92	1	194	0
10	2	87	1	189	0
25	2	72	1	174	0
50	0	0	0	149	0
100	0	0	0	99	0

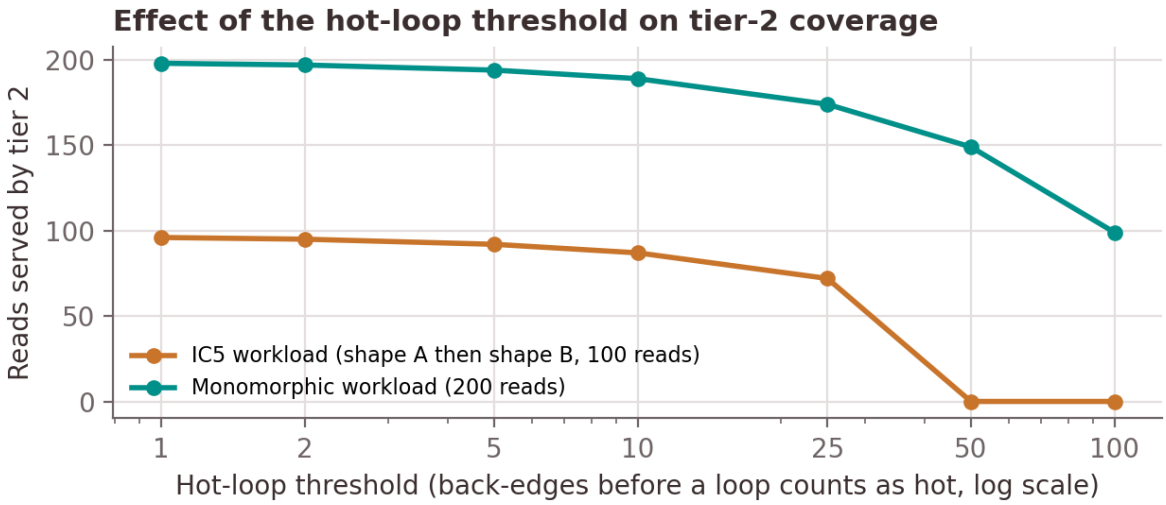


Figure 7: *Tier-2 coverage as a function of the hot-loop threshold*

Interpretation. A low threshold compiles earlier and serves more reads from tier 2. A high threshold has a subtler effect: on the IC5 workload, with a threshold of 50 or more the loop becomes hot only after shape B has appeared, so the site is already POLY and is *never* compiled — zero deopts, but also zero fast-path reads. Early compilation risks a deopt; late compilation risks missing the monomorphic window. The default of 5 compiles quickly while still letting the cache observe a few iterations. All 14 configurations produce 0 mismatches.

10.6 Comparison with V8

`v8_compare.js` runs the same mono/poly/mega experiment in V8 (Node v26.0.0): 50 × 1,000,000 reads of `o.x` with 1, 3 and 16 shapes. The polymorphic case took 1.69× and the megamorphic case 3.94× the monomorphic time (median of 3 runs). Absolute times are not comparable with our Python model, but the pattern is: V8 also handles one or a few shapes efficiently and slows down markedly at a megamorphic site — the behaviour our model predicts.

10.7 Overall assessment

- **Correctness:** 34/34 checks pass; zero mismatches in every benchmark, sweep and stress run; all seven JavaScript programs match V8 in all three modes.
- **Performance:** large gains where the lookup is expensive (2.09× on 200-field objects, 1.50× on 32-field objects with tier 2); the megamorphic weakness is fixed by capacity; small objects need tier 2 to break even.

11. Error Handling, Limitations and Known Issues

11.1 Error handling

Table 19: *Errors detected and how they are reported*

Condition	Detected by	Behaviour
Unexpected character, missing token	Lexer / parser	CompileError with line number, e.g. line 2: expected ')', got ';'
Undeclared variable	Parser	CompileError: 'y' is not defined
Unsupported construct (function, true, …)	Parser	CompileError: 'function' is not supported by this compiler
Division by zero	VM (DIV)	VMError: division by zero
Runaway loop	VM step counter	VMError: step limit exceeded (possible infinite loop)
Missing property	Baseline scan / IC miss	property 'y' not found on object
Unknown opcode	VM dispatch	VMError: unknown opcode
Node does not terminate	run_js.py	Reported as NODE TIMEOUT after 10 s

11.2 Unsupported features

- Strings, booleans (`true`/`false`), `null` and `undefined`.
- Functions, closures, `return`, classes, `this`, prototypes and `new`.
- Property writes (`p.x = 1`), element writes (`a[i] = 1`) and arrays of computed values; array literals must contain constants.
- `break`, `continue`, `switch`, `do…while`, exceptions; `console.log` with more than one argument.

11.3 Known behavioural differences from JavaScript

Table 20: *Known differences from JavaScript semantics*

Input	JavaScript (Node)	This engine	Reason
10 / 0	Infinity	VMError	Integer arithmetic only
7 / 2	3.5	3	DIV is integer division
-7 % 3	-1	2	% lowered via floor division
console.log(3 < 5)	true	1	Comparisons produce 1/0
p.missing	undefined	error	No undefined value
a[100] (out of range)	undefined	IndexError / wrong element	No bounds check on the object table

11.4 Technical constraints

- **Python timing:** speedups reflect fewer operations per read, not machine-code speed; interpreter overhead dominates small programs.

- **Tier 2 is a model:** the guarded load is a Python fast path, not generated machine code.
- **Only monomorphic sites are compiled;** polymorphic sites stay in the cache.
- **Shape registry is global** within a process, so shapes persist across tests; tests use distinct property names to avoid interference.
- **Objects are immutable** after creation except through `mutate_shape`, which exists only to test invalidation.

12. Member-Wise Contribution

Each member owned one module end to end — design, code and tests — and presents it in the demonstration and viva. The table below is the mandatory contribution matrix; the source locations listed can be inspected in the repository.

Table 21: *Final contribution matrix*

Member	Primary technical ownership	Deliverable produced	Evidence (files / tests)	Integration / support	Viva module
1 — Chilukuri Charith Reddy (24BCT0093)	Stack VM and baseline interpreter	Full 15-opcode interpreter; error handling (÷0, step limit)	VM.run() in vm_ic_model.py; TC1–TC10, TC12	System integration of VM with cache and tier 2	VM, ISA, fetch–decode–execute
2 — Vatsal Sharma (24BCE0334)	Profiler and hot-code detection; differential testing	Back-edge counters; configurable threshold	back_edge_counts / hot_flags; TC11; threshold_sweep.py	Differential-testing harness; timeline	Hot loops, threshold trade-off
3 — Mayan Manit (24BCE0506)	Hidden classes and inline cache	Shape/Obj; ICEntry; ic_getprop; MEGA table	IC1–IC4; benchmark section A; mega_tuning.py	Hit-rate and performance measurement	Cache states, invalidation, hit rates
4 — Ehsan Abdulkhader (24BCT0089)	Tier 2, deoptimisation, JS front end, evaluation	Guarded load + deopt; jsc.py; run_js.py; test_all.py; benchmark.py	VM.deoptimize(); IC5; examples/, tests/invalid/; repository commits	Repository, README, demo script, report and slides	Tier 2, deopt, compiler, results

The contribution log in Appendix E lists the commits in the shared repository.

13. Conclusion and Future Work

13.1 Objectives achieved

All seven objectives are met in the reference implementation. The engine executes bytecode correctly, detects hot loops, caches property offsets through four states, rejects stale entries after shape changes, compiles hot monomorphic reads to guarded loads and deoptimises safely, and runs real JavaScript programs whose output matches V8 in every mode. 34 of 34 regression checks pass and no benchmark, sweep or stress run produced a single mismatch.

13.2 Key learning

- **Correctness first.** A simple, trusted baseline and differential testing made every optimisation safe to build and easy to debug.
- **Caches have a cost.** An inline cache pays off only when the lookup it replaces is expensive; on small objects compilation is needed to win.
- **Capacity beats cleverness** for megamorphic sites: a better eviction policy did not help, more slots did.
- **Speculation needs a safety net.** Deoptimisation lets compiled code assume aggressively without ever producing a wrong answer, and timing (the hot threshold) decides whether speculation pays off.

13.3 Future work

1. **Native code generation:** emit the tier-2 guard and load as machine code (for example with asmjit) and measure native speedups.
2. **Polymorphic compilation:** compile POLY sites as a chain of guards instead of leaving them in the cache.
3. **Shared megamorphic cache:** replace per-site MEGA tables with one global table, as V8 does, and measure the memory–hit-rate trade-off.
4. **Larger language subset:** functions and calls (with call-site inline caches), strings, booleans and property writes with shape transitions.
5. **Realistic workloads:** benchmark programs with mixed site behaviour and skewed shape distributions, where LRU may outperform FIFO.

13.4 Publication and further-development potential

The project is well suited as an open educational artefact: a single-file, instrumented model of JIT techniques with a JavaScript front end and differential testing against V8. A short teaching paper or tool demonstration could report the measured trade-offs — cache cost on small objects, capacity versus policy, and the effect of the hot threshold on speculation — which are rarely shown quantitatively in teaching material.

14. References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson/Addison-Wesley, 2006.

[2] L. P. Deutsch and A. M. Schiffman, “Efficient implementation of the Smalltalk-80 system,” in *Proc. 11th ACM Symposium on Principles of Programming Languages (POPL)*, 1984, pp. 297–302.

[3] C. Chambers, D. Ungar and E. Lee, “An efficient implementation of SELF, a dynamically-typed object-oriented language based on prototypes,” in *Proc. OOPSLA ’89*, 1989, pp. 49–70.

[4] U. Hölzle, C. Chambers and D. Ungar, “Optimizing dynamically-typed object-oriented languages with polymorphic inline caches,” in *Proc. ECOOP ’91*, LNCS 512, Springer, 1991, pp. 21–38.

[5] U. Hölzle, C. Chambers and D. Ungar, “Debugging optimized code with dynamic deoptimization,” in *Proc. ACM PLDI ’92*, 1992, pp. 32–43.

[6] J. Aycock, “A brief history of just-in-time,” *ACM Computing Surveys*, vol. 35, no. 2, pp. 97–113, 2003.

[7] A. Gal et al., “Trace-based just-in-time type specialization for dynamic languages,” in *Proc. ACM PLDI ’09*, 2009, pp. 465–478.

[8] T. Würthinger et al., “One VM to rule them all,” in *Proc. Onward! 2013*, ACM, 2013, pp. 187–204.

[9] V8 project, “Fast properties in V8,” V8 blog, 2017. [Online]. Available: https://v8.dev/blog/fast-properties

[10] Node.js project, *Node.js documentation*. [Online]. Available: https://nodejs.org/docs/

[11] Python Software Foundation, *The Python Language Reference* and *Python Standard Library* (collections.OrderedDict, re). [Online]. Available: https://docs.python.org/3/

[12] Ecma International, *ECMA-262: ECMAScript Language Specification*. [Online]. Available: https://tc39.es/ecma262/

Acknowledgement of tools

Python and its standard library; Node.js/V8 as the reference engine; matplotlib for report figures; Microsoft PowerPoint and Word for the presentation and report; Git and GitHub for version control.

Appendix A: Installation and User Manual

A.1 Requirements

- Python 3.10 or newer (tested with 3.12.2). No third-party packages.
- Node.js (optional) for comparison with V8; without it, `run_js.py` compares our three modes with each other.

A.2 Getting the code

```
gh repo clone eak560/a4-bytecode-jit-inline-cache
cd a4-bytecode-jit-inline-cache
```

A.3 Commands

Table 22: *User commands*

Command	What it does	Time
`python3 test_all.py`	Full regression suite; exit status 0 if all pass	≈ 25 s
`python3 vm_ic_model.py`	TC1–TC12, IC1–IC5 and the synthetic benchmark	≈ 5 s
`python3 run_js.py examples/*.js`	Compile, run 3 ways, compare with Node	≈ 2 s
`python3 run_js.py FILE --bytecode`	Also print the generated bytecode	< 1 s
`python3 run_js.py FILE --max-steps N`	Lower the VM step limit	—
`python3 benchmark.py [--reps N]`	Median-of-N timings for the report	≈ 2 min
`python3 mega_tuning.py`	Megamorphic capacity × policy sweep	≈ 15 s
`python3 threshold_sweep.py`	Hot-threshold sweep	< 1 s
`python3 demo.py [--beat N]`	Paced presentation walkthrough	—
`./run_demo.command`	One-click live demo (Enter between steps)	≈ 1 min

A.4 Writing your own program

Create a `.js` file using the subset in Appendix C and run `python3 run_js.py myprog.js`. The report shows Node's output, whether each mode matches it, the timings, and the inline-cache state of every property-read site with its source line.

Appendix B: Sample Runs

B.1 run_js.py on examples/deopt.js

```
========================================================================
  examples/deopt.js
========================================================================
  compiled: 16 JS lines -> 25 bytecode instructions, 2 variables, 20 objects in 2 shape(s), 1 property-read
sites
  output: 10, 20, 30, 40, 50, 60 ...


                                  matches Node?
  our VM: baseline                yes                 —             (too short to time)
  our VM: + inline cache          yes                 —             (too short to time)
  our VM: + inline cache + tier 2 yes                 —             (too short to time)


  property-read call sites (inline-cache state at end of run):
    line 14  .x      POLY    2 shape(s) cached   | total += items[i].x;
  cache hits: 18 / 20  (90.0%)
  tier 2: 2 compile(s), 12 fast-path reads, 1 deopt(s)
```

B.2 Invalid input

```
$ python3 run_js.py tests/invalid/syntax_error.js
========================================================================
  tests/invalid/syntax_error.js
========================================================================
  COMPILE ERROR: line 2: expected ')', got ';'


$ python3 run_js.py tests/invalid/division_by_zero.js
  output: Infinity
  our VM: baseline                NO -- line 1: expected Infinity, got RUNTIME ERROR: division by zero
  our VM: + inline cache          NO -- line 1: expected Infinity, got RUNTIME ERROR: division by zero
  our VM: + inline cache + tier 2 NO -- line 1: expected Infinity, got RUNTIME ERROR: division by zero
```

Appendix C: Grammar of the Supported JavaScript Subset

```
program     := statement*
statement   := '{' statement* '}'
             | ('let' | 'const' | 'var') declaration ';'?
             | 'while' '(' expr ')' statement
             | 'for' '(' [decl | assign] ';' expr ';' [assign] ')' statement
             | 'if' '(' expr ')' statement ['else' statement]
             | 'console' '.' 'log' '(' expr ')' ';'?
             | assign ';'?
declaration := IDENT '=' (arrayLit | expr)
assign      := IDENT ('=' expr | '+=' expr | '-=' expr | '*=' expr | '++' | '--')
expr        := additive [('<'|'>'|'<='|'>='|'=='|'!='|'==='|'!==') additive]
additive    := term (('+' | '-') term)*
term        := unary (('*' | '/' | '%') unary)*
unary       := '-' unary | '!' unary | postfix
postfix     := IDENT '.' 'length' | IDENT '[' expr ']' ('.' IDENT)*   (IDENT is an
array)
             | primary ('.' IDENT)*
primary     := INT | IDENT | '(' expr ')' | objectLit
objectLit   := '{' [IDENT ':' const (',' IDENT ':' const)* [',']] '}'
arrayLit    := '[' [const (',' const)* [',']] ']'
const       := ['-'] INT | objectLit

Tokens:  INT = [0-9]+   IDENT = [A-Za-z_$][A-Za-z0-9_$]*   comments // and /* */
```

Appendix D: Complete Test Table

Table 23: *All regression checks from test_all.py*

#	Group	Test	Expected	Actual	Result
1	unit	TC1	PASS	PASS	PASS
2	unit	TC2	PASS	PASS	PASS
3	unit	TC3	PASS	PASS	PASS
4	unit	TC4	PASS	PASS	PASS
5	unit	TC5	PASS	PASS	PASS
6	unit	TC6	PASS	PASS	PASS
7	unit	TC7	PASS	PASS	PASS
8	unit	TC8	PASS	PASS	PASS
9	unit	TC9	PASS	PASS	PASS
10	unit	TC10	PASS	PASS	PASS
11	unit	TC11	PASS	PASS	PASS
12	unit	TC12	PASS	PASS	PASS
13	unit	IC1	PASS	PASS	PASS
14	unit	IC2	PASS	PASS	PASS
15	unit	IC3	PASS	PASS	PASS
16	unit	IC4	PASS	PASS	PASS
17	unit	IC5	PASS	PASS	PASS
18	end-to-end	basics.js	14 lines = Node	all 3 modes match	PASS
19	end-to-end	deopt.js	20 lines = Node	all 3 modes match	PASS
20	end-to-end	edge_cases.js	9 lines = Node	all 3 modes match	PASS
21	end-to-end	points_mega.js	1 lines = Node	all 3 modes match	PASS
22	end-to-end	points_mono.js	1 lines = Node	all 3 modes match	PASS
23	end-to-end	points_poly.js	1 lines = Node	all 3 modes match	PASS
24	end-to-end	points_wide.js	1 lines = Node	all 3 modes match	PASS
25	invalid	division_by_zero.js	runtime error (division by zero)	runtime error: division by zero	PASS
26	invalid	infinite_loop.js	runtime error (step limit exceeded)	runtime error: step limit exceeded (possible infinite loop)	PASS
27	invalid	missing_property.js	runtime error (property 'y' not found)	runtime error: property 'y' not found on object	PASS
28	invalid	syntax_error.js	compile error (expected ')')	compile error: line 2: expected ')', got ';'	PASS
29	invalid	undefined_variable.js	compile error ('y' is not defined)	compile error: line 2: 'y' is not defined	PASS

#	Group	Test	Expected	Actual	Result
30	invalid	unsupported_function.js	compile error ('function' is not supported)	compile error: line 2: 'function' is not supported by this compiler	PASS
31	invalid	unsupported_string.js	compile error (unexpected character)	compile error: line 2: unexpected character '"'	PASS
32	stress	mono 50,000 reads	0 mismatches	0 mismatches	PASS
33	stress	poly 50,000 reads	0 mismatches	0 mismatches	PASS
34	stress	mega 50,000 reads	0 mismatches	0 mismatches	PASS

Appendix E: Contribution Log (Repository History)

Commits in the shared repository at the time of writing (`git log`):

```
d35920c  2026-10-23  eak560  Static contents page with real page numbers; clean
benchmark run
a3e32b4  2026-10-23  eak560  Final Review 3 report and its generator
9458b7a  2026-10-23  eak560  Regression suite, negative tests, benchmark and
threshold sweep
f425204  2026-10-23  eak560  One-click demo script (run_demo.command)
17fbe7d  2026-10-23  eak560  Final presentation script matching the final deck
ec71e25  2026-10-23  eak560  Final presentation and megamorphic cache tuning
f1f4088  2026-10-23  eak560  A4 bytecode VM with inline caching, JS front end and
Review 2 materials
```